Week 2 - Testing in pipelines (unit, integration, load and security testing stages)

DOP-C02 Domain 1 (SDLC Automation). The exam wants you to know **which test runs where**, **which AWS service or feature runs it**, **how a failed test stops the pipeline**, and **how results are reported**. The core rule is **shift left**: cheap, fast tests early; slow, expensive tests later in production-like environments.

Part 1 - The testing pyramid mapped to pipeline stages

```
fewer, slower, costlier
   Production:  canary + synthetic monitoring + alarms
   Staging:     load / performance, DAST, chaos (FIS), UAT + manual approval
   Test env:    integration, API / contract, end-to-end UI
   Build:       unit tests, lint, SAST, SCA, secrets scan, IaC scan, image scan
   Local / PR:  pre-commit hooks, unit tests, PR checks
more, faster, cheaper
```

Test type	What it checks	Where it runs	Typical AWS tool
Static checks (lint, format)	Code style and obvious errors	Pre-commit and build	CodeBuild
Unit	Single functions or classes, dependencies mocked	**Build stage**	CodeBuild + **test reports** + **coverage reports**
SAST	Insecure code patterns	Build stage / PR	Amazon Q Developer code review, Amazon Inspector code scanning, SonarQube, Semgrep (CodeGuru Reviewer / Security are legacy)
SCA	Vulnerable **dependencies**	Build stage	Amazon Inspector (SBOM scan), OWASP Dependency-Check, Snyk
Secrets scanning	Keys or passwords in code	Pre-commit, PR, build	git-secrets, trufflehog, CodeGuru Reviewer secrets detector
IaC scanning	Insecure or invalid templates	Build stage	**cfn-lint**, **cfn-guard**, cfn-nag, Checkov; CloudFormation **Guard Hooks**
Container image scanning	OS and package CVEs	After image build, before deploy	**ECR basic / enhanced scanning (Inspector)**, Inspector CI/CD scan
Integration	Components working together with real AWS resources	**Test environment**, after deploy	CodeBuild running tests against the deployed stack
Contract / API	Request and response compatibility	Test environment	Pact, Postman/Newman in CodeBuild
End-to-end / UI	User flows in a browser or app	Test or staging	Selenium / Playwright in CodeBuild, **AWS Device Farm**
Load / performance	Throughput, latency, scaling, limits	**Staging** (production-like)	**Distributed Load Testing on AWS**, JMeter, k6, Locust
DAST	Vulnerabilities in the **running** app	Staging	OWASP ZAP, Burp
Chaos / resilience	Behaviour under failure	Staging, carefully in production	**AWS Fault Injection Service (FIS)** with stop conditions
Smoke / synthetic	Key paths work after deploy	Every environment, continuously in production	**CloudWatch Synthetics canaries**, CodeDeploy hooks
User acceptance	The business signs off	Staging	**Manual approval action**

Part 2 - How tests run and fail the pipeline

CodeBuild is the test runner

- Any buildspec command that returns a **non-zero exit code** fails the phase: the build **fails**, the pipeline action **fails**, and the stage **stops**.
- `on-failure: ABORT` (default) stops the build. `on-failure: CONTINUE` keeps going.
- `finally` blocks run even after a failure (publish reports, clean up).
- **Test reports: JUnit XML**, NUnit, NUnit3, TestNG, **Cucumber JSON**, Visual Studio TRX. Shown per **report group** with pass/fail counts and trends.
- **Code coverage reports: JaCoCo XML**, **Cobertura XML**, Clover XML, SimpleCov JSON, **lcov**. Line and branch coverage.
- Report groups keep data **30 days**. **Export the raw files to S3** to keep them longer.
- Report status events go to **EventBridge** for notifications.
- **CodeBuild doesn't fail the build on low coverage by itself.** Make the test tool exit non-zero, for example `pytest --cov-fail-under=80` or JaCoCo `check` rules.

```
version: 0.2
phases:
  install:
    runtime-versions:
      python: 3.12
  build:
    commands:
      - pip install -r requirements.txt
      - pytest tests/unit --junitxml=reports/unit.xml --cov=app --cov-report=xml:reports/coverage.xml
    --cov-fail-under=80
  post_build:
    commands:
      - echo "tests done"
reports:
  unit-tests:
    files: [unit.xml]
    base-directory: reports
    file-format: JUNITXML
  coverage:
    files: [coverage.xml]
    base-directory: reports
    file-format: COBERTURAXML
```

Running tests faster

- **Batch builds:** a **matrix** (several runtimes or OS types) or a **graph** (dependencies between builds), in parallel.
- **Parallel test execution (test splitting):** shard one suite across several builds (`codebuild-tests-run` helper).
- **Caching:** local or S3 cache for dependencies.
- **CodeBuild local agent** to test buildspecs on your machine.

CodePipeline test stages and gates

Feature	How it is used for testing
Test action category (provider CodeBuild)	Runs test commands as a pipeline action
Parallel actions (same `runOrder`)	Unit, SAST and lint at the same time. **The stage fails if any of them fails**
Sequential `runOrder`	Deploy to test first, then run integration tests
Commands action	Simple shell commands without a separate CodeBuild project
Lambda invoke action	Custom checks. The function must call `PutJobSuccessResult` / `PutJobFailureResult`
Manual approval action	UAT or security sign-off. SNS notification. Times out after **7 days**

Feature	How it is used for testing
V2 stage conditions	`beforeEntry` / `onSuccess` / `onFailure` rules: **CloudWatch alarm rule**, Lambda rule, VariableCheck rule, deployment window
Stage failure handling	`onFailure` **rollback** to the last successful run, or **retry** the failed actions
Third-party test actions	AWS Device Farm (mobile and browser), BlazeMeter, Ghost Inspector

Stage order: fail fast

1. Source, then build with **unit tests, lint, SAST, SCA, secrets and IaC scans in parallel**, then build and **scan the image**.
2. Deploy to **test**, then **integration and API tests**.
3. Deploy to **staging**, then **load, DAST and E2E tests**, then **manual approval**.
4. Deploy to **production** with canary or blue/green, **alarms** and **synthetic canaries**; rollback on failure.

Part 3 - Integration testing patterns

Ephemeral (per-run) test environments

1. A CloudFormation, SAM or CDK action **creates a stack** named after the run, for example `app-test-${CODEBUILD_BUILD_NUMBER}`.
2. CodeBuild **reads the stack outputs** (API URL, table name) through CodePipeline **output variables** (`#{TestDeploy.ApiUrl}`) or `aws cloudformation describe-stacks`.
3. Run the integration tests against those real resources.
4. **Delete the stack** in a `finally` block or a later action.

- **Benefits:** a clean environment each run, no shared-state conflicts, parallel feature branches.

Test data and isolation

- **Separate AWS accounts** for test, staging and production (AWS Organizations), so tests never touch production data.
- **Seed data:** fixtures, or **restore an RDS snapshot** with masked or anonymised data, never real customer PII.
- **Mocks and emulators:** LocalStack, DynamoDB Local, `sam local invoke` for fast build-stage tests.
- **Secrets for tests:** CodeBuild `env: secrets-manager:` or `parameter-store:`, never in the buildspec.

Testing during deployment (hooks)

Service	Hook for tests
CodeDeploy EC2	`ValidateService` script (smoke test after `ApplicationStart`)
CodeDeploy ECS	`AfterAllowTestTraffic` (tests through the **test listener**), `BeforeAllowTraffic`
CodeDeploy Lambda / SAM	`BeforeAllowTraffic` **/ PreTraffic**, `AfterAllowTraffic` / PostTraffic
Elastic Beanstalk	Test the green environment URL before **swap URLs**
EKS	`kubectl rollout status`, Argo Rollouts analysis steps

Part 4 - Load and performance testing

- **Where:** a **production-like staging** environment (same size and configuration). **Not production**, unless carefully controlled.
- **Distributed Load Testing on AWS** (AWS Solutions): runs **JMeter, k6, Locust or Taurus** scripts on **Fargate tasks** across Regions, thousands of simulated users, results in a dashboard and S3, callable from a pipeline through its API.
- **Simple pipeline load test: k6 or JMeter in CodeBuild** with **thresholds** (for example p95 latency under 300 ms, error rate under 1%). A failed threshold makes the tool exit non-zero, failing the action.
- **Watch** CloudWatch metrics: latency, 5XX, throttles, CPU, DB connections, Lambda concurrency. Watch **service quotas**: load tests often hit limits such as Lambda concurrency or API Gateway throttling.
- **Profiling: CodeGuru Profiler** for hotspots, **X-Ray** for slow downstream calls.
- Don't load test production on every commit: use staging, on a schedule (for example nightly) or before release. Follow AWS testing policies; simulated DDoS stress tests have their own policy.

Part 5 - Security testing in the pipeline (DevSecOps)

Check	Tool	How it gates the pipeline
Secrets in code	git-secrets (pre-commit), trufflehog, CodeGuru Reviewer secrets detector	Pre-commit hook blocks the commit, or the build fails
SAST	Amazon Q Developer code review, Amazon Inspector code scanning, SonarQube, Semgrep	Build fails on high-severity findings
SCA	Amazon Inspector **SBOM** scanning (`inspector-sbomgen` + **Inspector Scan API**), OWASP Dependency-Check, Snyk	Build fails on critical CVEs
IaC	**cfn-lint** (syntax, best practice), **cfn-guard** (policy-as-code rules), cfn-nag, Checkov; **CloudFormation Guard Hooks**	Build fails on violations; a hook blocks the stack operation
Container image	**ECR enhanced scanning (Inspector)** (continuous, OS **and** language packages), ECR basic scanning (on push, OS only), **Inspector CI/CD scan** before push	Script checks `describe-image-scan-findings` and fails on CRITICAL, or EventBridge scan event to Lambda
Image signing	**AWS Signer** with Notation	Deploy-time verification blocks unsigned images
DAST	OWASP ZAP in CodeBuild against **staging**	Build fails on high-risk alerts
Runtime and account	GuardDuty, Security Hub, AWS Config rules	Detect and alert; feeds automation

- **Scan before deploy** and **fail on CRITICAL**. Don't just scan and forget.
- **ECR basic scanning = OS packages only. Enhanced (Inspector)** adds **language packages** and **continuous re-scans** when new CVEs are published.
- **Least privilege:** the test CodeBuild role has no production permissions; test in a **separate account**.

Part 6 - After deployment (testing in production safely)

- **CloudWatch Synthetics canaries:** scripted API and UI checks (Puppeteer or Selenium) **every few minutes**, with alarms on success percentage. Use as a **post-deploy smoke test** and as an **alarm for automatic rollback** (CodeDeploy alarms, ECS alarm rollback, CodePipeline V2 rule).
- **Canary or linear deployments with alarms** (CodeDeploy, SAM, ECS, Lambda): real traffic is the final test.
- **AWS FIS (chaos engineering):** experiment templates (stop instances, add latency, throttle API calls, kill ECS tasks, AZ power interruption). **Stop conditions are CloudWatch alarms**. Run in staging first; a pipeline stage can start an experiment.
- **Feature flags (AppConfig)** and **CloudWatch RUM** limit and observe the blast radius of new features.

Part 7 - Exam traps

Trap	Truth
CodeBuild fails the build when coverage is below 80%	Coverage reports **only show** coverage. Enforce it in the test tool (`--cov-fail-under`, JaCoCo rules) so it **exits non-zero**
Tests failed but the pipeline continued	The exit code was swallowed by an always-true fallback or a later succeeding command (see note below), or `on-failure: CONTINUE` was set
Keep test reports for a year in CodeBuild	Report groups keep **30 days**. **Export raw files to S3**
A Lambda invoke action hangs until timeout	The function must call `PutJobSuccessResult` / `PutJobFailureResult`
Load test production on every commit	Use a **production-like staging** environment, on a schedule or before release
Integration tests use production data	**Separate accounts** and masked or synthetic data
ECR basic scanning finds vulnerable npm packages	Basic = OS only. **Enhanced (Inspector)** = OS **and** language packages, continuous re-scans
Scan the image after it is in production	Scan **in the pipeline, before deploy**, and **fail on CRITICAL**
cfn-lint enforces our security rules	cfn-lint checks syntax and best practice. Use **cfn-guard** or **Guard Hooks** for your own rules
Test database passwords in buildspec variables	Use `env: secrets-manager:` / `parameter-store:`
One parallel test action fails and the pipeline moves on	The **stage fails** and the next stage doesn't run
Stop a chaos experiment if the error rate spikes	FIS **stop conditions** (CloudWatch alarms)
Test a new ECS version before production traffic	CodeDeploy **test listener** + **AfterAllowTestTraffic**
Users must sign off before production	**Manual approval** action (7-day timeout, SNS)
Block promotion if an alarm is firing	CodePipeline **V2 stage condition** with a **CloudWatch alarm rule**
A long test suite slows every build	**Batch builds / test splitting**, caching, move slow tests later

- **Note on swallowed exit codes:** a test command ending with `|| true`, or a line where a later command succeeds, hides the failure from CodeBuild.

Part 8 - Hands-on lab (about 90 minutes, low cost)

Lab 1 - Unit tests with reports and a coverage gate

1. Create a small Python app with `app.py` and `tests/test_app.py`, including one failing test.
2. Create a CodeBuild project with the buildspec from Part 2 (JUnit and Cobertura reports, `--cov-fail-under=80`).
3. Run it: the build **fails**. Open **Reports** to see the failed test and the coverage.
4. Fix the test and run again. The report group shows the trend.
5. Append `|| true` to the pytest command: the build passes despite failures. **That is the trap.** Revert it.

Lab 2 - A pipeline with parallel checks

1. Create a CodePipeline (V2): Source, Build, Test.
2. In Test, add three CodeBuild actions with the **same runOrder**: unit tests; `cfn-lint template.yaml` plus `cfn-guard validate -d template.yaml -r rules.guard` (rule: S3 buckets must be encrypted); `git secrets --scan` (or trufflehog).
3. Commit a template with an unencrypted bucket. **Only** the guard action fails, but the **stage fails**.

Lab 3 - An ephemeral integration environment

1. Add a CloudFormation deploy action creating `app-test` (SAM API Gateway + Lambda) with output `ApiUrl` and namespace `TestDeploy`.
2. Add a CodeBuild action with `API_URL = #{TestDeploy.ApiUrl}`; run `curl -f $API_URL/health` and a small pytest suite.
3. Add a final action that deletes the stack (or delete it in the buildspec `finally` block).

Lab 4 - A load test with thresholds

1. In CodeBuild, install k6 and run a script with thresholds `http_req_duration: ['p(95)<300']` and `http_req_failed: ['rate<0.01']` against the **test** API.
2. Lower the threshold to `p(95)<5`: the action fails because of k6's exit code.
3. Watch Lambda Duration and Throttles in CloudWatch during the test.

Lab 5 - An image scan gate

1. Turn on **ECR enhanced scanning** for a repository and push an old image (for example based on `python:3.8`).
2. In CodeBuild after the push, wait for the scan and fail on CRITICAL:

```
aws ecr wait image-scan-complete --repository-name app --image-id imageTag=$TAG
COUNT=$(aws ecr describe-image-scan-findings --repository-name app --image-id imageTag=$TAG \
  --query 'imageScanFindings.findingSeverityCounts.CRITICAL' --output text)
[ "$COUNT" = "None" ] || [ "$COUNT" -eq 0 ] || { echo "Critical CVEs: $COUNT"; exit 1; }
```

Lab 6 (optional) - Synthetics and an alarm rule

1. Create a **CloudWatch Synthetics** heartbeat canary against the test API, with an alarm on SuccessPercent below 100.
2. Add a CodePipeline V2 **beforeEntry** condition on the Prod stage with a **CloudWatch alarm rule**. Break the API and see promotion **blocked**.

Clean up

- Delete the stacks, pipelines, CodeBuild projects, ECR images, the canary and alarms, and the S3 artifact buckets.

Part 9 - Check yourself

Questions

1. Where should unit tests, SAST and secrets scanning run, and why?
2. The build must fail if code coverage drops below 80%. Coverage reports are already set up. What else is needed?
3. Test reports must be kept for 1 year for auditors. What do you do?
4. A Lambda invoke action running custom API checks always ends at the action timeout. Why?
5. Integration tests need the API URL of a stack the pipeline just deployed. How do you pass it?
6. ECR basic scanning shows no findings, but a vulnerable npm library is in the image. Why, and what is the fix?
7. Security wants every S3 bucket in CloudFormation templates to be encrypted, checked in the pipeline. Which tool?
8. Where and how should load tests run, and how do they fail the pipeline?
9. You must test a new ECS version through a separate port before production traffic shifts. What do you use?
10. A chaos experiment must stop automatically if the 5XX rate rises. How?
11. Production promotion must be blocked while a staging CloudWatch alarm is in ALARM. Which feature?
12. A test stage has three parallel actions and one fails. What happens?

Answers

1. In the **build stage, early** (or pre-commit / PR): fast and cheap, catching problems before anything is deployed (shift left, fail fast).
2. Make the test tool **exit non-zero below the threshold** (for example `pytest --cov-fail-under=80`). Coverage reports only display results.
3. Report groups keep **30 days**. **Export the raw report files to S3**, with lifecycle rules if needed.
4. The function didn't call `PutJobSuccessResult` / `PutJobFailureResult`.
5. **CodePipeline output variables** from the CloudFormation action (namespace plus `#{Namespace.Output}`) as CodeBuild environment variables, or `describe-stacks`.
6. Basic scanning covers **OS packages only**. Use **ECR enhanced scanning (Inspector)** for language packages and continuous re-scans, or Inspector SBOM / CI scanning in the build.
7. **cfn-guard** in CodeBuild and/or **CloudFormation Guard Hooks** at deploy time. cfn-lint is for syntax.
8. In a **production-like staging** environment (Distributed Load Testing on AWS, or k6 / JMeter in CodeBuild). **Thresholds make the tool exit non-zero**. Watch metrics and service quotas.
9. **CodeDeploy ECS blue/green** with a **test listener** and the **AfterAllowTestTraffic** hook.
10. **AWS FIS stop conditions** with a CloudWatch alarm on the 5XX rate.
11. A **CodePipeline V2 stage condition** (`beforeEntry`) with a **CloudWatch alarm rule**.
12. The others still finish, but the **stage fails** and the pipeline **doesn't continue**. V2 can **retry or roll back on failure**.

Quick memory summary

- **Shift left, fail fast:** unit, lint, SAST, SCA, secrets, IaC and image scans in the build stage (in parallel); integration in a test env; load, DAST, E2E and UAT in staging; canary, synthetics and alarms in production.
- **CodeBuild:** non-zero exit fails the build; test reports (JUnit, Cucumber, TestNG...) and coverage reports (JaCoCo, Cobertura, lcov...); **30-day** retention, export to S3; enforce coverage in the test tool; batch builds and test splitting.
- **CodePipeline:** test actions, parallel actions (one fails, stage fails), Lambda invoke (PutJobSuccessResult / PutJobFailureResult), manual approval (7 days), **V2 conditions with alarm rules**, rollback or retry on failure.
- **Integration:** ephemeral stacks, output variables, separate accounts, masked data, secrets from Secrets Manager.
- **Load:** staging, Distributed Load Testing on AWS or k6 / JMeter with thresholds, watch quotas.
- **Security:** ECR **enhanced** (Inspector) for language packages, fail on CRITICAL before deploy, **cfn-guard** for policy rules, ZAP for DAST, Signer for image signing.
- **Production:** Synthetics canaries, deployment alarms, **FIS** with alarm stop conditions.